项目全仓库深度分析报告

项目名称: Literature Report Agent (PaperReader)
分析日期: 2026-04-12
分析视角: AI Agent 系统架构审查 + 工程实践评估

---

1. 项目概览

1.1 项目定位

PaperReader 是一个面向学术文献调研场景的 AI Agent 系统，核心定位是**多智能体协作的文献综述报告生成引擎**。用户输入研究主题（中文/英文），系统自动完成需求澄清 → 搜索规划 → 检索抽取 → 报告生成 → 质量审核的全流程，最终输出结构化 Markdown 调研报告。

1.2 业务背景

学术文献调研是 AI+学术场景的典型高价值场景。传统方式依赖人工检索、阅读、整理，效率低且容易遗漏。该项目试图用 Multi-Agent 系统模拟人类研究员的工作流：

- Planner（规划者）：理解用户意图，拆解研究问题
- Retriever（检索者）：多源检索论文
- Analyst（分析师）：抽取关键信息、对比分析
- Reviewer（审核者）：验证引用真实性、检查报告质量

1.3 目标用户

- 科研人员：快速了解某个研究领域的现状
- 学生：撰写文献综述或课程报告
- AI Agent 工程师：将本项目作为 Multi-Agent + RAG 系统架构参考

1.4 解决的问题

传统方式的问题	本项目解决的点
人工检索效率低	多源并行检索（SearXNG + arXiv API）
论文信息提取耗时	LLM 批量抽取 PaperCards
引用来源不可追溯	citation resolve → claim verify 全链路 grounding
单次报告质量无保障	Review 节点 + grounding 审核
无法持续优化	完整 eval 体系 + RAG 评测框架

1.5 为什么它不是普通脚本/问答/RAG demo

维度	普通 RAG demo	本项目
工作流	单次检索-生成	7 节点 StateGraph + 多 Agent 协作
状态管理	无状态	AgentState 跨节点状态传递
质量保障	无	Review + Claim Verification 双重保障
评测体系	无	完整 RAG 评测（Layer 1-4 metrics）
可观测性	无	NodeEventEmitter + TraceStore
多 Agent 协作	无	Supervisor 编排 + role-based dispatch
可恢复性	无	Task snapshot 持久化

1.6 仓库实现成熟度判断

阶段：Phase 3 末 / Phase 4 早期

- [x] 核心工作流已跑通（research graph 7 节点）
- [x] 评测体系已建立（scifact 数据集 + 4 层指标）
- [x] Multi-Agent 编排骨架已实现（AgentSupervisor + role dispatch）
- [x] MCP 基础设施已落地（MCPAdapter + stdio/HTTP transport）
- [x] Memory 系统已实现（三层架构）
- [ ] 并发 DAG fan-out 尚未实现（search/extract 内部并行化）
- [ ] Agent 间 peer-to-peer 通信尚未实现
- [] 外部 MCP server 实际接入尚未完成
- [] Phase 4 评测体系尚未建立

---

2. 前沿工程共识总结

2.1 成熟 Agent 系统应具备的要素

基于对 OpenAI Agents SDK、Anthropic Engineering Blog、LangGraph 官方文档的扫描，成熟 Agent 系统通常需要：

要素	说明	当前项目
Orchestration	多步工作流编排（Graph/DAG）	✅ LangGraph StateGraph
Reasoning	思考链、re-plan、reflection	✅ ClarifyAgent + repair 策略
Tool Calling	结构化工具注册、调用、超时、重试	✅ ToolRuntime + MCPAdapter
MCP	标准化工具/资源/提示接入协议	✅ MCPAdapter（实现中）
Skills	可复用技能包（prompt bundle/workflow）	✅ SkillsRegistry
Memory/State	感官/工作/长期记忆分层管理	✅ MemoryManager（三层）
Structured Output	Pydantic schema 绑定输出	✅ 多处使用 Pydantic v2
Lifecycle Control	启动/暂停/恢复/取消	⚠️ 基础实现（terminate endpoint）
Harness Engineering	Runtime harness、中间件、事件追踪	✅ NodeEventEmitter + TraceStore
Eval/Benchmark	retrieval/reviewer/workflow 多层评测	✅ RagEvalRunner（4层）
Trace/Observability	执行链路可追踪	✅ /trace endpoint + trace_store

2.2 行业主流共识

1. **Graph over Agent Loop**：用 DAG 替代无限循环的 agent loop，提高可预测性和可调试性
2. **Specialist Agents > Monolithic Agent**：按角色拆分（Planner/Retriever/Analyst/Reviewer），各司其职
3. **Grounded Generation**：生成结果必须能溯源到具体证据，citation grounding 是标配
4. **Eval-Driven Development**：评测先行，通过量化指标驱动迭代优化
5. **MCP 标准化**：Model Context Protocol 正在成为 Agent 工具接入的事实标准

2.3 分析判断标准

本报告后续章节采用以下判断标准：

- **已实现**：有完整代码、API、测试覆盖
- **部分实现**：有骨架但未完成或有明显缺口
- **规划中**：仅有文档或占位代码
- **缺失**：完全未涉及

---

3. 仓库结构与主工作流总览

3.1 目录结构

```
src/
├── api/                    # FastAPI 应用层
│   ├── app.py              # 主应用入口
│   └── routes/
│       ├── tasks.py        # Task 生命周期 API（/tasks）
│       ├── corpus_search.py
│       ├── corpus_evidence.py
│       ├── evals.py
│       ├── agents.py       # Agent dispatch API
```

```
│       ├── config.py       # 模型配置 API
│       ├── mcp.py          # MCP 管理 API
│       ├── skills.py       # Skills API
│       └── workspaces.py   # Workspace artifacts API
├── agent/                  # Agent 核心（CLI、LLM 封装）
│   ├── cli.py              # 交互式 CLI 入口
│   ├── llm.py              # LLM client（OpenAI-compatible）
│   ├── settings.py         # 全局配置（Settings 单例）
│   ├── prompts.py          # Prompt 模板
│   ├── callbacks.py        # LangChain callbacks
│   ├── report.py           # 报告渲染
│   └── report_frame.py     # 报告结构化框架
├── graph/                  # Report Workflow（Phase 1-3 主工作流）
│   ├── state.py            # AgentState TypedDict
│   ├── builder.py          # build_report_graph() — 12 节点
│   ├── callbacks.py        # NodeEventEmitter
│   ├── instrumentation.py  # instrument_node 装饰器
│   └── nodes/              # 12 个节点实现
│       ├── input_parse.py
│       ├── ingest_source.py
│       ├── extract_document_text.py
│       ├── normalize_metadata.py
│       ├── classify_paper_type.py
│       ├── retrieve_evidence.py
│       ├── draft_report.py
│       ├── report_frame.py
│       ├── survey_intro_outline.py
│       ├── repair_report.py
│       ├── resolve_citations.py
│       ├── verify_claims.py
│       ├── apply_policy.py
│       └── format_output.py
├── research/               # Research Workflow（Phase 1-3 调研工作流）
│   ├── graph/
│   │   ├── builder.py      # build_research_graph() — 7 节点
│   │   └── nodes/
│   │       ├── clarify.py       # Clarify 节点
│   │       ├── search_plan.py   # SearchPlan 节点
│   │       ├── search.py        # Search 节点
│   │       ├── extract.py       # Extract 节点
│   │       ├── draft.py         # Draft 节点
│   │       ├── review.py        # Review 节点
│   │       └── persist_artifacts.py  # Persist 节点
│   ├── agents/             # V2 Multi-Agent 实现
│   │   ├── supervisor.py    # AgentSupervisor（编排核心）
│   │   ├── clarify_agent.py # ClarifyAgent（ResearchBrief 生成）
│   │   ├── planner_agent.py  # PlannerAgent
│   │   ├── retriever_agent.py # RetrieverAgent
│   │   ├── analyst_agent.py  # AnalystAgent
│   │   └── reviewer_agent.py # ReviewerAgent
│   ├── policies/           # Policy 层
│   │   ├── clarify_policy.py  # Clarify 策略
```

```
│   │   └── search_plan_policy.py
│   ├── prompts/            # Prompt 模板
│   │   ├── clarify_prompt.py
│   │   └── search_plan_prompt.py
│   ├── services/           # Service 层
│   │   ├── grounding.py
│   │   └── reviewer.py
│   └── research_brief.py   # ResearchBrief 数据模型
├── corpus/                 # 语料库管理
│   ├── ingest/             # 文档解析与 chunking
│   │   ├── loaders.py
│   │   ├── parsers.py
│   │   ├── coarse_chunker.py
│   │   ├── fine_chunker.py
│   │   ├── canonicalize.py
│   │   └── pipeline.py
│   ├── search/             # 检索引擎
│   │   ├── retrievers/
│   │   │   ├── paper_retriever.py   # PaperRetriever
│   │   │   ├── chunk_retriever.py   # ChunkRetriever
│   │   │   ├── keyword_retriever.py # KeywordRetriever
│   │   │   └── dense_retriever.py   # DenseRetriever
│   │   ├── deduper.py
│   │   ├── reranker.py
│   │   ├── candidate_builder.py
│   │   └── result_builder.py
│   └── store/              # 持久化存储
│       ├── repository.py   # CorpusRepository
│       ├── document_store.py
│       ├── chunk_store.py
│       ├── vector_index.py # FAISS
│       └── keyword_index.py
├── tools/                  # 工具层
│   ├── registry.py         # ToolRuntime（全局工具注册表）
│   ├── specs.py            # ToolSpec 定义
│   ├── search_tools.py     # SearXNG 搜索工具
│   ├── arxiv_api.py        # arXiv API 工具
│   ├── arxiv_paper.py
│   ├── web_fetch.py
│   ├── pdf.py
│   ├── local_fs.py
│   ├── rag_search.py
│   └── mcp_adapter.py      # MCPAdapter（MCP 客户端）
├── skills/                 # Skills 系统
│   ├── registry.py         # SkillsRegistry
│   ├── orchestrator.py     # Skill 编排器
│   ├── discovery.py        # Skills 文件系统发现
│   ├── runner.py
│   └── research_skills.py  # 内置科研技能
├── eval/                   # 评测系统
│   └── rag/
│       ├── runner.py      # RagEvalRunner
```

```
│        ├── metrics.py      # 4 层评测指标
│        ├── models.py       # 评测数据模型
│        ├── report.py       # 评测报告生成
│        ├── matchers.py
│        └── scifact_corpus.py # SciFact 数据集加载器
├── verification/           # 引用验证
│    ├── claim_judge.py
│    ├── reachability.py
│    └── source_tiers.py
├── db/                     # 数据库层
│    ├── engine.py          # SQLAlchemy engine
│    ├── models.py          # ORM 模型（PostgreSQL）
│    └── task_persistence.py # Task 快照持久化
├── memory/                 # Memory 系统
│    ├── manager.py         # MemoryManager（三层架构）
│    └── store.py
├── models/                 # 数据模型（纯 Pydantic）
│    ├── task.py
│    ├── report.py
│    ├── paper.py
│    ├── agent.py
│    ├── config.py
│    ├── eval.py
│    ├── mcp.py
│    ├── skills.py
│    ├── workspace.py
│    ├── review.py
│    └── trace.py
├── embeddings/             # Embedding 服务
│    └── client.py          # get_embedding_client()
├── retrieval/              # 检索服务
│    ├── search.py
│    └── citations.py
├── ingest/                 # 文档导入
│    ├── db.py
│    └── ingestor.py
├── tasking/                # 任务运行时
│    └── trace_wrapper.py   # TraceStore
└── verification/           # 验证服务

frontend/
├── src/
│    ├── App.tsx            # 主应用
│    ├── components/
│    │    ├── TaskSubmitForm.tsx   # 任务提交
│    │    ├── TaskHistory.tsx      # 任务历史
│    │    ├── Phase34Panel.tsx     # Phase 3/4 面板
│    │    ├── ReportPreview.tsx     # 报告预览
│    │    ├── ChatPanel.tsx         # 交互式对话
│    │    ├── ReviewFeedbackPanel.tsx # 审核反馈
│    │    ├── SkillPalette.tsx      # Skill 面板
│    │    ├── ConfigPanel.tsx      # 配置面板
```

```
│   │   ├── ModelConfigPanel.tsx  # 模型配置
│   │   ├── SessionOverview.tsx   # Session 总览
│   │   ├── GraphView.tsx         # 图可视化
│   │   ├── TraceTimeline.tsx     # Trace 时间线
│   │   ├── AgentPanel.tsx        # Agent 面板
│   │   ├── ThinkingPanel.tsx     # 思考过程
│   │   └── ToolLogPanel.tsx     # 工具日志
│   ├── hooks/
│   │   └── useTaskDetail.ts
│   └── types/
│       ├── task.ts
│       └── phase34.ts
```

3.2 核心入口

入口	路径	说明
API	`src/api/app.py`	FastAPI 应用，通过 Uvicorn 启动
CLI	`src/agent/cli.py`	交互式 CLI（`python -m src.agent.cli`）
任务提交	`POST /tasks`	创建 Research/Report 任务
Research Graph	`src/research/graph/builder.py::build_research_graph()`	7 节点调研工作流
Report Graph	`src/graph/builder.py::build_report_graph()`	12 节点报告工作流

3.3 主流程

Research Workflow（调研工作流）

```
用户输入研究主题
    │
    ▼
clarify (ClarifyAgent)
    │  生成 ResearchBrief (topic / goal / sub_questions)
    ▼
search_plan (SearchPlanAgent)
    │  生成 SearchPlan (query_groups)
    ▼
search (SearXNG 并行查询)
    │  产出 paper_candidates (rag_result)
    ▼
extract (LLM 批量抽取 PaperCards)
    │  每批 3 篇，并行抽取
    ▼
draft (DrafterAgent)
    │  综合 PaperCards → DraftReport + draft_markdown
    │  失败时回退到模板生成
    ▼
review (ReviewerAgent)
    │  grounding: resolve_citations → verify_claims
    │  产出 ReviewFeedback
    ▼
```

```
persist_artifacts
    │  持久化所有 artifact 到 workspace
    ▼
输出：resolved_report + verified_report + final_report
```

Report Workflow（报告生成工作流）

```
input_parse → ingest_source → extract_document_text → normalize_metadata
    │
    ▼
retrieve_evidence → classify_paper_type
    │
    ├──→ draft_report（draft 模式）
    ├──→ report_frame（full 模式）
    └──→ survey_intro_outline（survey 模式）
    │
    ▼
repair_report（损坏报告修复）
    │
    ▼
resolve_citations → verify_claims → apply_policy → format_output
```

3.4 关键模块职责

模块	职责	核心文件
`research.graph`	调研工作流编排	`builder.py`（7 节点 DAG）
`research.agents`	V2 Multi-Agent 实现	`supervisor.py`（编排器）
`tools`	工具注册与执行	`registry.py`（ToolRuntime）
`mcp_adapter`	MCP 协议客户端	`mcp_adapter.py`（Adapter）
`skills`	Skills 注册与执行	`registry.py`（SkillsRegistry）
`corpus.search`	检索引擎	`retrievers/`（4 种 Retriever）
`eval.rag`	RAG 评测体系	`runner.py`（RagEvalRunner）
`memory`	三层记忆系统	`manager.py`（MemoryManager）
`db`	PostgreSQL 持久化	`models.py`（ORM）+ `task_persistence.py`
`tasking`	Trace 可观测性	`trace_wrapper.py`（TraceStore）
`api.routes.tasks`	任务生命周期	`tasks.py`（ThreadPoolExecutor 执行）

---

4. 业务需求与 Story 设计分析

4.1 业务场景

核心场景：学术文献调研报告生成

用户输入：`"调研医疗垂类大模型中的 AI agent 发展"`

系统输出：

1. Research Brief（topic / goal / sub-questions）
2. Search Plan（多组查询策略）
3. Paper Cards（每篇论文的结构化抽取结果）
4. Draft Report（Markdown 格式初步报告）
5. Review Feedback（质量审核意见）
6. Final Report（grounded 报告 + 引用溯源）

4.2 用户路径

```
1. 前端提交研究主题 → POST /tasks（source_type=research）
2. 后台 ThreadPoolExecutor 执行 Research Graph
3. 前端轮询 /tasks/{id} 或 SSE 订阅 /tasks/{id}/events
4. 各节点状态实时更新（clarify → search_plan → search → ...）
5. 完成后查询 /tasks/{id}/result 获取最终报告
6. 可选：通过 /tasks/{id}/chat 继续对话
7. 可选：通过 /tasks/{id}/trace 查看执行链路
```

4.3 Story 完整性评估

Story	状态	说明
研究主题 → 结构化报告	✅ 跑通	完整 7 节点工作流
多源论文检索	✅ 跑通	SearXNG + arXiv API
论文信息自动抽取	✅ 跑通	LLM 批量抽取 PaperCards
引用溯源	✅ 跑通	resolve_citations → verify_claims
报告质量审核	✅ 跑通	ReviewerAgent + grounding
任务持久化	✅ 跑通	PostgreSQL snapshot
可恢复执行	⚠️ 部分	AgentSupervisor 有 re-plan，但未完整测试
Multi-Agent 并行	⚠️ 内部并行	search/extract 内部并行化，但无 DAG fan-out
Skill 编排	⚠️ 部分	SkillsRegistry 已实现，但 orchestrator 有限
Workspace artifacts	⚠️ 部分	已持久化，但 artifact schema 尚未标准化

闭环任务链：已形成完整闭环（clarify → search → extract → draft → review → persist），但 re-plan / revise 链路尚未完全打通。

5. Agent 架构与 Workflow 设计分析

5.1 单 Agent 还是 Multi-Agent

当前：混合架构

- **Phase 1-3 Report Workflow**：12 节点单 Graph，每个节点是一个函数（不等同于 Agent）
- **Phase 4 Research Workflow**：7 节点 Graph + AgentSupervisor 编排多个 V2 Agent
- **V2 Agent 实现**：PlannerAgent / RetrieverAgent / AnalystAgent / ReviewerAgent 已注册到 Supervisor

5.2 Workflow / Graph / Orchestrator / Scheduler 组织

组件	实现	文件
Report Graph	LangGraph StateGraph（12 节点）	`src/graph/builder.py`
Research Graph	LangGraph StateGraph（7 节点）	`src/research/graph/builder.py`
Orchestrator	AgentSupervisor	`src/research/agents/supervisor.py`
Dispatch	role-based（PLANNER/RETRIEVER/ANALYST/REVIEWER）	`supervisor.py::dispatch()`
Scheduler	ThreadPoolExecutor（max_workers=4）	`api/routes/tasks.py`

5.3 Reasoning / Planning / Re-plan

能力	实现	说明
Clarify（意图理解）	ClarifyAgent	4 策略降级：structured_output → json_parse → repair → limited
SearchPlan（搜索规划）	SearchPlanAgent	生成 query_groups
Re-plan	AgentSupervisor.replan()	按 stage 裁剪 state，重新执行
快速路径	_fast_path_brief()	简单 topic 用 policy heuristics

评估：Planning 层已实现，但 re-plan 触发条件较简单（仅基于 `needs_followup` 和 `review_passed`），缺乏动态 re-plan 机制（如工具调用失败、检索结果不足时的自动 re-plan）。

5.4 Review / Revise / Loop

- **Review**：ReviewerAgent + `resolve_citations → verify_claims` grounding 链路
- **Revise**：仅在 `repair_report` 节点有模板回退，无真正的 revise loop
- **Loop**：仅在 clarify 有 `needs_followup` 循环；review 通过后直接 persist，无 revise 循环

5.5 "真正 Agent" 还是 "Workflow + Tool Calling"

判断：Workflow + Tool Calling，带有早期 Multi-Agent 特征

当前系统的工作流节点更接近"函数式 pipeline step"，而非"自主决策 Agent"。真正的 Agent 应具备：

- 自主决定下一步行动（不依赖预定义 DAG 顺序）
- 基于观察动态调整计划
- 工具选择自主决策

当前系统虽然有 AgentSupervisor 和 V2 Agents，但执行仍以 DAG 顺序为主。**这是一个精心设计的自动化 Pipeline，而非真正的自主 Agent**。这本身不是缺点——对于文献调研这类结构化任务，DAG + 专家分工是合理的设计。

---

6. Agent 技术细节分析

6.1 Reasoning

ClarifyAgent 的多策略推理：

```
# 策略降级链：structured_output → json_parse → repair → limited
async def run(input):
    brief = _try_structured_output(...)   # OpenAI 原生结构化输出
    if brief is None:
        brief = _try_json_parse(raw_text)  # JSON 解析 + Pydantic 验证
    if brief is None:
        brief = _try_repair(bad_output)     # 二次 LLM 修复
    if brief is None:
        brief = to_limited_brief(raw_query) # 保底
```

Fast Path：`_fast_path_brief()` 对简单 topic 使用 policy heuristics 跳过 LLM 调用，提升响应速度。

6.2 Plan / Execute / Re-plan

方法	文件	说明
SearchPlanAgent	`research/agents/search_plan_agent.py`	生成 query_groups
AgentSupervisor.replan()	`research/agents/supervisor.py`	按 stage 裁剪 + 重新执行
_prune_state_for_stage()	`research/agents/supervisor.py`	保留 upstream state

6.3 Tools 调用方式

工具注册：`src/tools/registry.py` — ToolRuntime

```
class ToolRuntime:
    def register_function(self, name, func, spec: ToolSpec):
        self._functions[name] = func
        register_tool(spec)

    def invoke(self, tool_name, **kwargs) -> ToolResult:
        # 同步调用 + 计时 + 统一错误处理
```

工具 Schema：`src/tools/specs.py` — ToolSpec + get_tool_spec() + register_tool()

MCP 工具：`src/tools/mcp_adapter.py` — MCPServerInstance.invoke()

```python
async def _invoke_tool(self, name: str, args: dict):
    resp = await self._transport.send("tools/call", {"name": name, "arguments": args})
    return resp.get("result", {})
```

6.4 State 管理

层级	实现	文件
Graph State	AgentState TypedDict	`src/graph/state.py`
Task State	TaskRecord	`src/models/task.py`
Workspace State	WorkspaceModel	`src/models/workspace.py`
Memory State	MemoryManager	`src/memory/manager.py`

AgentState 关键字段：

```python
class AgentState(TypedDict):
    raw_input: str
    source_type: Literal["arxiv", "pdf", "research"]
    report_mode: Literal["draft", "full"]
    brief: dict | None
    search_plan: dict | None
    rag_result: Any | None
    paper_cards: list[Any]
    draft_report: DraftReport | None
    resolved_report: ResolvedReport | None
    verified_report: VerifiedReport | None
    final_report: FinalReport | None
    review_feedback: Any | None
    review_passed: bool | None
    tokens_used: Annotated[int, operator.add]  # 累加
    warnings: Annotated[list[str], operator.add]  # 累加
    errors: Annotated[list[str], operator.add]  # 累加
    node_statuses: dict[str, NodeStatus]
```

6.5 输出结构化程度

多层次 Pydantic 模型：

层级	模型	文件
调研简报	ResearchBrief	`src/research/research_brief.py`
搜索计划	SearchPlan	多个 policy/model 中
检索结果	RagResult	`src/models/research.py`
论文卡片	PaperCard	`src/models/research.py`
报告框架	ReportFrame	`src/models/report.py`
草稿报告	DraftReport	`src/models/report.py`
解析报告	ResolvedReport	`src/models/report.py`
验证报告	VerifiedReport	`src/models/report.py`
最终报告	FinalReport	`src/models/report.py`
审核反馈	ReviewFeedback	`src/models/review.py`

6.6 Review / Reflection / Verification

Review 节点：`src/research/graph/nodes/review.py`

执行链路：

1. `resolve_citations` — 将报告中的引用映射到具体 evidence
2. `verify_claims` — 验证每个 claim 的 grounding 状态（supported / partial / ungrounded / abstained）
3. `format_output` — 生成最终 Markdown

Grounding 评估（`src/eval/rag/metrics.py`）：

- Layer 1：Paper Retrieval（Recall@K）
- Layer 2：Evidence Retrieval（F1 / NDCG）
- Layer 3：Citation Reachability
- Layer 4：Claim Support（supported / partial / unsupported）

---

7. Context Engineering 分析

7.1 Prompt Engineering

组件	文件	说明
Clarify System Prompt	`src/research/prompts/clarify_prompt.py`	包含 few-shot examples
Clarify User Prompt	`build_clarify_user_prompt()`	动态组装输入
SearchPlan Prompt	`src/research/prompts/search_plan_prompt.py`	查询策略生成
Report Prompts	`src/agent/prompts.py`	报告生成 prompt
Chat System Prompts	`src/agent/report_frame.py`	REGULAR_CHAT / SURVEY_CHAT

多语言支持：报告默认输出中文（Markdown），但底层 LLM 支持中英文。

7.2 RAG

四层检索架构（来自 `docs/current-architecture-and-usage.md`）：

```
Layer 1: Paper Retrieval (PaperRetriever)
    └── keyword_retriever + dense_retriever → candidate_merger

Layer 2: Dedup + Rerank
    └── PaperDeduper → CrossEncoderReranker

Layer 3: Evidence Retrieval (ChunkRetriever)
    └── 细粒度 chunk 检索 (coarse → fine)

Layer 4: Citation Grounding
    └── resolve_citations → verify_claims
```

RAG 评测（`src/eval/rag/runner.py`）：

- 支持策略对比：`hybrid_basic` / `keyword_only` / `dense_only`
- 自动加载 scifact 数据集
- 4 层量化指标

7.3 Memory

三层记忆架构（`src/memory/manager.py`）：

```
┌──────────────────────────────────────────────────┐
│  SensoryMemory (感官记忆)                          │
│  → 当前轮次工具输出、用户输入、外部信号              │
│  → 环形缓冲 (50 事件) 、TTL=当前会话                 │
├──────────────────────────────────────────────────┤
│  WorkingMemory (工作记忆)                          │
│  → 当前会话 LLM 上下文窗口内容                       │
│  → agent 循环状态 (iter_count)                     │
│  → context_budget (token 预算)                     │
├──────────────────────────────────────────────────┤
│  LongTermMemory (长期记忆)                         │
│  → SemanticMemory: 向量语义检索 (JSON 存储)         │
│  → EpisodicMemory: 跨会话情景记忆 (任务轨迹)        │
│  → PreferenceMemory: 用户偏好                      │
└──────────────────────────────────────────────────┘
```

评估：Memory 系统已完整实现，但存储使用 JSON 文件（`.memory/semantic/`），未接入 PostgreSQL。跨会话记忆持久化有文件依赖。

7.4 State / History

- **Graph State**：AgentState TypedDict，跨节点传递

- **Task History**：`task.chat_history`（最近 8 轮对话）
- **Node Event History**：`task.node_events`（SSE 推送）
- **Trace History**：`TraceStore`（/trace endpoint）

7.5 Structured Outputs

广泛使用 Pydantic v2：

- 所有 API 请求/响应模型
- 所有业务数据模型（Brief / PaperCard / Report 等）
- ClarifyAgent 支持 `with_structured_output()` + 降级策略

7.6 Context Loading / Passing / Compaction

- **Context 压缩**：`_fast_path_brief()` 跳过简单 topic 的 LLM 调用
- **Context 注入**：`MemoryManager.inject_into_messages()` 自动追加到 system prompt
- **Context 预算**：`WorkingMemory.context_budget = 6000`（硬编码）
- **State 传递**：LangGraph Annotated[int, operator.add] 累加 tokens_used / warnings / errors

7.7 上下文设计体系评估

已形成体系：

- Prompt 模板化 + 动态组装
- RAG 四层检索架构
- 三层 Memory 系统
- Pydantic schema 强制结构化
- Context 注入到 LLM 的标准接口

缺口：

- Context 压缩（contextual summary）未实现
- Memory 未接入 PostgreSQL（文件存储）
- Chunking 策略缺乏自适应机制

---

8. Multi-Agent 设计分析

8.1 Orchestrator（编排层）

AgentSupervisor（`src/research/agents/supervisor.py`）：

```
class AgentSupervisor:
    # 编排决策：选择 legacy graph node 还是 v2 agent
    async def run_node(self, node_name, state, inputs):
        backend_mode = self._get_backend_mode(canonical)
        if backend_mode == LEGACY:
            result = await self._run_legacy(...)
        else:
            result = await self._run_v2(...)
```

```
        return result

    # 协作主循环
    async def collaborate(self, state, start_node, stop_after):
        for node_name in workflow:
            result = await self.run_node(node_name, state, payload)
            self._merge_state(state, result)
            if self._should_stop_after(node_name, state):
                break

    # Role-based dispatch
    async def dispatch(self, role: AgentRole, state, inputs):
        node_name = ROLE_DEFAULT_NODES[role]
        return await self.run_node(node_name, state, inputs)
```

Router / Controller：`AgentSupervisor._infer_start_node()` 实现断点恢复逻辑，根据已有 artifact 推断起始节点。

8.2 Workers（工作者）

Worker	Role	核心职责	Paradigm
ClarifyAgent	意图理解	4 策略生成 ResearchBrief	Structured Output
PlannerAgent	搜索规划	生成 query_groups	Plan-and-Execute
RetrieverAgent	信息检索	多源检索 + metadata 获取	TAG
AnalystAgent	分析综合	PaperCards → DraftReport	Reasoning via Artifacts
ReviewerAgent	质量审核	Grounding + Claim Verification	Reflexion

角色边界：各 Agent 职责明确（各负责一个 artifact type），但 **Agent 间无 peer-to-peer 通信**，所有状态通过 AgentState 传递。

8.3 Shared State / Artifacts

共享对象	持久化位置	说明
ResearchBrief	`task.brief`	Clarify 输出
SearchPlan	`task.search_plan`	Planner 输出
RagResult	`task.rag_result`	Retriever 输出
PaperCards	`task.paper_cards`	Analyst 输入
DraftReport	`task.draft_report`	Analyst 输出
ReviewFeedback	`task.review_feedback`	Reviewer 输出
Workspace Artifacts	PostgreSQL (`artifacts` 表)	最终持久化

统一 Schema：`src/models/` 下所有 Pydantic 模型

8.4 Multi-Agent 成熟度评估

维度	成熟度	说明
Orchestrator	⚠️ 部分	有 Supervisor，但缺乏动态调度
Workers	✅ 基础	各 Agent 职责明确，但无真正自主决策
Shared State	✅ 良好	AgentState 统一传递，schema 标准化
Agent 间通信	❌ 缺失	无 peer-to-peer，仅通过 state 传递
并行 Agent	⚠️ 内部并行	search/extract 内部并行，无 DAG fan-out
动态重分配	❌ 缺失	Agent 角色固定，无动态重分配

结论：当前是 **"精心组织的 Pipeline"**，而非成熟 Multi-Agent 系统。Agent 间分工良好，但缺乏真正的自主协作、动态调度和 Agent 间直接通信。

---

9. Function Calling / Tools 分析

9.1 Tool Schema

ToolSpec（`src/tools/specs.py`）：

```
class ToolSpec(TypedDict):
    name: str
    description: str
    category: str
    input_schema: dict  # JSON Schema
    tags: list[str]
```

9.2 Tool Registry / Tool Runtime

ToolRuntime（`src/tools/registry.py`）：

- 全局单例：`get_runtime()`
- 函数注册：`register_function(name, func, spec)`
- 统一调用：`invoke(tool_name, **kwargs)` → ToolResult
- 工具列表：`list_registered()`

9.3 Function Calling Loop

当前：同步顺序调用

```python
# search_node 内部（SearXNG 并行）
results = ThreadPoolExecutor(max_workers=min(8, len(all_queries))).map(
    lambda q: _search_searxng(q), all_queries
)

# MCP 异步调用
async def _invoke_tool(self, name, args):
    resp = await self._transport.send("tools/call", {...})
    return resp.get("result", {})
```

9.4 输入输出约束

ToolResult 统一结构：

```python
class ToolResult(TypedDict):
    ok: bool
    output: str | None
    error: str | None
    tool_name: str
    latency_ms: float | None
```

9.5 Timeout / Retry / Fallback

机制	实现	说明
工具级超时	⚠️ 未显式实现	MCP transport 有 timeout=30s
重试	❌ 未实现	ToolRuntime 无 retry 机制
Fallback	✅ 部分	ClarifyAgent 有 4 策略降级
降级模式	✅ 有	`degradation_mode: normal/limited/safe_abort`

9.6 Trace / Observability

- **NodeEventEmitter**：节点开始/结束/错误事件
- **TraceStore**：`src/tasking/trace_wrapper.py`（NodeRun + ToolRun）
- **SSE 事件流**：`/tasks/{id}/events`
- **Trace API**：`/tasks/{id}/trace`

9.7 权限 / Auth / 调用边界

- ToolRuntime 无权限控制（所有注册工具均可调用）
- API 端点无显式鉴权（仅 .env 配置）
- 无工具级别的调用频率限制

---

10. MCP / Skills 分析

10.1 MCP 实现

MCPAdapter（`src/tools/mcp_adapter.py`）：

组件	说明
MCPTransport	抽象传输层
StdioTransport	stdio 启动本地 MCP server
RemoteHttpTransport	HTTP 连接远程 MCP server
MCPServerInstance	单个 server 的运行时（tools/prompts/resources discover）
MCPAdapter	全局管理器（start/stop/invoke）

MCPAdapter API（`src/api/routes/mcp.py`）：

- `GET /mcp/servers` — 列出已注册 server
- `POST /mcp/servers` — 注册新 server
- `POST /mcp/servers/{id}/start` — 启动 server
- `POST /mcp/servers/{id}/invoke` — 调用 tool
- `GET /mcp/servers/{id}/tools` — 列出 tools

MCP 数据模型（`src/models/mcp.py`）：

- MCPServerConfig
- MCPToolDescriptor / MCPPromptDescriptor / MCPResourceDescriptor
- MCPInvocationRequest / MCPInvocationResponse
- MCPInvokeKind（TOOL / PROMPT / RESOURCE）

评估：MCP 适配器基础设施已完成，支持 stdio 和 HTTP 两种 transport，但**实际外部 MCP server 接入尚未完成**。

10.2 Skills 实现

SkillsRegistry（`src/skills/registry.py`）：

Backend 类型	Handler	说明
LOCAL_GRAPH	LocalGraphHandler	调用 local LangGraph 节点
LOCAL_FUNCTION	LocalFunctionHandler	调用 Python 函数
MCP_TOOLCHAIN	MCPToolchainHandler	通过 MCP 调用工具链
MCP_PROMPT	MCPPromptHandler	通过 MCP 获取 prompt

内置 Skills（8 个）：

1. `research_lit_scan` — 多源论文检索
2. `paper_plan_builder` — 报告大纲生成

3. `creative_reframe` — 主题重塑
4. `workspace_policy_skill` — 工作区策略注入
5. `lit_review_scanner` — 文献综述扫描
6. `claim_verification` — 声明验证
7. `comparison_matrix_builder` — 对比矩阵构建
8. `experiment_replicator` — 实验复现分析
9. `writing_scaffold_generator` — 写作脚手架生成

Skills 生命周期：

1. 发现：`discover_from_filesystem()` 扫描 `.agents/skills/` 和 `.claude/skills/`
2. 注册：`register(SkillManifest)`
3. 配置：`set_handlers()` 注入 backend handler
4. 执行：`run(SkillRunRequest)` → handler.run()

Skills API（`src/api/routes/skills.py`）：

- `GET /skills` — 列出所有 skills
- `POST /skills/{id}/run` — 执行 skill

评估：Skills 系统骨架完整，支持 4 种 backend，懒加载 + progressive disclosure 设计合理。实际 skill 数量较少（9 个内置），skills 与 workspace/artifacts 对接的完整性待验证。

---

11. Lifecycle Control 与 Harness Engineering 分析

11.1 Lifecycle Control

操作	实现	文件
Launch	`POST /tasks` → `_schedule_task_run()`	`api/routes/tasks.py`
Run	ThreadPoolExecutor 执行 graph	`tasks.py::_bg_executor`
Pause	❌ 未实现	—
Resume	`_infer_start_node()` 按已有 artifact 推断起点	`supervisor.py`
Retry	❌ 未实现（仅有 graceful degradation）	—
Cancel	`POST /tasks/{id}/terminate`（标记 + 中止轮询）	`tasks.py`
State Serialization	PostgreSQL snapshot	`task_persistence.py`

断点恢复机制：

```python
def _infer_start_node(self, state: dict) -> str:
    if not state.get("brief"):
        return "clarify"
    if not state.get("search_plan"):
        return "search_plan"
    if not state.get("rag_result"):
        return "search"
    # ... 依次判断各 artifact 是否存在
```

评估：有基础的 lifecycle control（create/run/terminate），但 pause/resume/retry 未实现。cancel 仅标记状态，不强制终止执行线程。

11.2 Async Continuation / Webhook / Callback

- **Async**：ThreadPoolExecutor + `asyncio.to_thread()` 包装
- **SSE**：`/tasks/{id}/events` — Server-Sent Events 流式推送节点状态
- **Webhook**：❌ 未实现

11.3 Harness Engineering

组件	实现	文件
Runtime Harness	ThreadPoolExecutor + task isolation	`tasks.py::_bg_executor`
Middleware / Wrapper	`instrument_node()` 装饰器	`graph/instrumentation.py`
Middleware / Wrapper	`NodeEventEmitter` 事件发射	`graph/callbacks.py`
Long-running Agent	TaskRecord + SSE 状态推送	`models/task.py`
Session / Context 持久化	PostgreSQL snapshot + chat_history	`db/task_persistence.py`
Trace / Event Logging	TraceStore + node_events	`tasking/trace_wrapper.py`
Lifecycle API Control	`/terminate` endpoint	`api/routes/tasks.py`

评估：Harness 层实现较完善，有事件发射、节点仪表化、状态持久化。缺口在于缺乏 pause/resume/retry 机制，cancel 无法强制终止线程。

---

12. 代码设计分析

12.1 模块划分

分层清晰：

```
API 层（routes/）→ Agent 层（research/agents/）→ Graph 层（graph/）→ Tools/Corpus 层
```

每层职责明确，无明显的跨层直接调用（除了明确的依赖关系）。

12.2 核心抽象

抽象	实现	评价
StateGraph	LangGraph StateGraph	✅ 成熟框架
ToolRuntime	全局工具注册表	✅ 清晰简单
SkillsRegistry	Skill 注册 + 多 backend handler	✅ 可扩展
MemoryManager	三层 Memory 统一接口	✅ 设计完整
RagEvalRunner	评测执行器	✅ 策略模式
AgentSupervisor	编排器	⚠️ 功能完整但复杂

12.3 Schema / Model / Storage / Service / API

- **Schema/Pydantic**：`src/models/`（10+ 个模型文件）
- **Storage/DB**：`src/db/`（SQLAlchemy ORM + PostgreSQL）
- **Service**：`src/research/services/`（grounding / reviewer）
- **API**：`src/api/routes/`（10+ 个路由文件）

12.4 耦合度

高内聚、低耦合：

- 每层独立模块，跨层依赖通过接口
- Graph 节点通过 AgentState 通信，无直接依赖
- Tools 通过 ToolRuntime 注册，无硬编码

部分耦合问题：

- `api/routes/tasks.py` 过于庞大（1100+ 行），混合了状态管理、图执行、API 路由
- `research/graph/builder.py` 引用了 `AgentState`（应该通过 type import 而非运行时 import）
- `_run_graph_sync` 和 `_run_graph` 两个函数重复逻辑

12.5 可维护性

优点：

- 每个节点独立文件，修改影响范围小
- Pydantic 模型提供 IDE 类型提示
- 文档较完整（README + docs/）

问题：

- 部分文件过长（tasks.py 1100+ 行）
- 重复代码：`_run_graph_sync` 和 `_run_graph` 逻辑重复
- 部分节点实现较简单（模板回退），缺乏测试覆盖

12.6 扩展性

易扩展点：

- 添加新节点：只需在 builder 中注册
- 添加新 Tool：在 registry 中注册
- 添加新 Skill：在 manifest 中注册
- 添加新 Retriever：继承 base class

难扩展点：

- 修改 AgentState：涉及所有节点
- 修改 Graph 拓扑：涉及 builder 和所有 routing 函数

---

13. 量化指标 / Eval / Benchmark 分析

13.1 Retrieval Eval

RagEvalRunner（`src/eval/rag/runner.py`）执行流程：

```
1. Paper Retrieval (PaperRetriever) → predicted_papers
2. Dedup + Rerank → deduped/reranked papers
3. Evidence Retrieval (ChunkRetriever) → predicted_chunks
4. Citation Projection → citations
5. compute_all_metrics() → 4 层指标
```

评测数据集：

- `tests/eval/cases/phase2_smoke.jsonl` — 冒烟测试集
- `tests/eval/cases/scifact_full.jsonl` — SciFact 完整数据集

13.2 Reviewer Eval

Claim Verification（`src/verification/claim_judge.py`）：

- Claim 分类：supported / partial / ungrounded / abstained
- Source Tiers：L1（直接引用）/ L2（相关引用）/ L3（弱相关）
- Reachability 分析：`src/verification/reachability.py`

13.3 Workflow Eval

当前：无专门的 workflow-level 评测

- 各节点有单元测试（如 `tests/research/graph/nodes/test_review_node.py`）
- E2E 测试（`tests/test_e2e.py` / `tests/test_e2e_clarify.py`）
- 集成测试（`tests/test_integration_cli.py`）

13.4 Benchmark / Regression / Smoke

类型	文件	说明
Smoke	`phase2_smoke.jsonl`	冒烟测试集
Regression	`scifact_regression.jsonl`	回归测试集
Full	`scifact_full.jsonl`	完整评测集

版本对比：评测报告支持 strategy comparison mode（多策略对比）

13.5 Golden Set / Eval Cases

- **Golden Set**：`scifact` 学术引用验证数据集（110 个 claim-paper 对）
- **Eval Cases**：手工构建的 `phase2_smoke.jsonl`

13.6 评测指标（Layer 1-4）

Layer	指标	说明
L1	Recall@K	Paper Retrieval 召回率
L2	F1 / NDCG	Evidence Retrieval 质量
L3	Citation Reachability	引用可达性
L4	Claim Support	supported / partial / unsupported

13.7 评测体系评估

已实现：

- ✅ 4 层检索评测体系
- ✅ 策略对比模式
- ✅ 评测报告自动落盘
- ✅ 评测用例持久化

缺口：

- ❌ 无 workflow-level 评测（clarify / draft quality 评测）
- ❌ 无 LLM-as-judge 主观评测
- ❌ 无 benchmark 版本对比（回归告警）
- ❌ Phase 4 评测体系尚未建立

迭代指导能力：评测结果可指导检索策略选择（hybrid vs keyword vs dense），但无法指导 Agent 行为优化。

---

14. 软件工程与软件测试分析

14.1 测试覆盖

类型	文件数	覆盖率
Unit Test	~45 个文件	各模块独立测试
Integration Test	`test_integration_cli.py`	CLI 端到端
E2E Test	`test_e2e.py` / `test_e2e_clarify.py`	Research workflow
Smoke Test	`phase2_smoke.jsonl`	核心路径
Regression Test	`scifact_regression.jsonl`	检索回归

14.2 测试方法论

方法	实现	说明
Mock / Stub	⚠️ 部分	LLM 调用有 mock，但不够系统
Fixtures	✅ 有	`tests/conftest.py`
Golden Dataset	✅ 有	`scifact_full.jsonl`
Property-based	❌ 未实现	—
AST-based analysis	❌ 未实现	—
Mutation testing	❌ 未实现	—

14.3 CI Test Gates

当前：无显式 CI 配置（无 `.github/workflows/` 或 `Makefile` test target）

14.4 Flaky Tests

当前：无 flaky test 检测机制

14.5 Deterministic Replay

Trace 可回放：

- `TraceStore` 记录 NodeRun / ToolRun
- `/tasks/{id}/trace` 返回完整执行链路
- 但无基于 trace 的 deterministic replay 测试

14.6 测试体系评估

类型	状态	说明
Unit Tests	✅ 良好	~45 个测试文件，覆盖核心模块
Integration Tests	✅ 有	CLI + API 集成测试
E2E Tests	⚠️ 基础	2 个 E2E 文件，覆盖有限
Smoke Tests	✅ 有	phase2_smoke.jsonl
Golden Tests	✅ 有	scifact 数据集
CI/CD	❌ 缺失	无 GitHub Actions 或 Makefile
Mutation Testing	❌ 缺失	—
Flaky Test Detection	❌ 缺失	—

优先补足：

1. CI/CD pipeline（GitHub Actions）
2. E2E 测试覆盖（当前 2 个太少）
3. Mutation testing 基础框架

---

15. 面试价值总结

15.1 最值得讲的技术亮点

1. **Multi-Agent 协作架构**：Supervisor 编排 + role-based dispatch + 断点恢复
2. **RAG 四层检索体系**：Paper → Dedup/Rerank → Evidence → Citation Grounding
3. **Grounding 评测链路**：resolve_citations → verify_claims → 4 层量化指标
4. **三层 Memory 系统**：Sensory / Working / Long-term 的学术标准定义
5. **ClarifyAgent 多策略降级**：structured_output → json_parse → repair → limited
6. **MCP 基础设施**：stdio/HTTP 双传输层 + tools/prompts/resources discover
7. **Skills 懒加载系统**：progressive disclosure + 多 backend handler

15.2 最容易被追问的地方

亮点	潜在追问
Multi-Agent 编排	"Agent 间怎么通信？是无状态还是有状态协作？"
RAG 四层架构	"Layer 3 和 Layer 4 怎么区分？evidence reachability 怎么算？"
Grounding 评测	"claim verification 的 ground truth 怎么构建？"
三层 Memory	"跨会话记忆怎么持久化？用的是向量数据库还是 PG？"
Clarify 降级	"4 策略的 fallback 顺序怎么确定的？"
MCP 实现	"实际接入了哪些 MCP server？"

15.3 最容易被质疑的地方

1. **"这不是真正的 Multi-Agent"**：Agent 间无 peer-to-peer 通信，仅通过 state 传递
2. **"并行化不够"**：search/extract 内部并行，但无 DAG fan-out
3. **"评测体系不完整"**：workflow-level 评测缺失，Phase 4 评测空白
4. **"cancel 不彻底"**：terminate endpoint 仅标记状态，无法强制终止线程
5. **"Memory 存储用文件"**：三层 Memory 存 JSON 文件，未接入 PostgreSQL

15.4 如何更稳地表述这个项目

不要说	应该说
"我们实现了 Multi-Agent 系统"	"我们实现了基于 Supervisor 的多角色协作工作流"
"Agent 间可以自由通信"	"各 Agent 通过统一 AgentState 共享状态"
"系统支持动态并行"	"search 和 extract 节点内部实现了并行检索"
"评测体系完整"	"RAG 评测覆盖了 retrieval 4 层指标，workflow 评测在规划中"
"支持 MCP"	"MCP 适配器已完成，支持 stdio/HTTP 传输"

---

16. 未来优化建议

16.1 Workflow / Agent Architecture

1. **引入 DAG fan-out**：search → extract 阶段支持并发执行多个查询组
2. **增强 Supervisor**：支持动态 agent 重分配（基于任务类型/负载）
3. **添加 Revise Loop**：review 失败时触发 revise 节点，形成反馈闭环
4. **完善 Re-plan 机制**：基于工具调用失败/检索结果不足自动触发 re-plan

16.2 Context Engineering

1. **Context 压缩**：实现 contextual summary，缓解 context 窗口压力
2. **自适应 Chunking**：根据查询类型动态选择 chunk size

3. **Memory 持久化**：将 SemanticMemory / EpisodicMemory 迁移到 PostgreSQL

16.3 Multi-Agent

1. **Agent 间通信**：引入消息队列或共享事件总线，支持 peer-to-peer
2. **动态 Role 分配**：基于任务特征动态选择/组合 Agent
3. **Agent 生命周期**：为每个 Agent 添加独立的启动/监控/回收机制

16.4 Tools / Function Calling

1. **重试机制**：ToolRuntime 添加 retry + exponential backoff
2. **超时控制**：为每个工具添加可配置 timeout
3. **权限控制**：引入工具级别权限或 sandbox

16.5 MCP / Skills

1. **接入外部 MCP server**：接入实际工具（如文件系统、GitHub）
2. **Skill 版本管理**：支持 skill 升级/回滚
3. **Skill 评测**：为每个 skill 建立评测用例

16.6 Lifecycle / Harness

1. **强制 Cancel**：实现真正的线程终止（而非仅标记）
2. **Pause / Resume**：支持任务暂停和恢复
3. **Webhook**：添加任务完成/失败 Webhook 回调

16.7 Eval / Benchmark

1. **Workflow 评测**：建立 clarify quality / draft quality / review accuracy 评测
2. **LLM-as-Judge**：引入主观评测（coherence / relevance / creativity）
3. **Regression Alert**：CI 中集成评测，失败时告警
4. **Phase 4 评测**：建立 multi-agent 协作效率评测

16.8 Software Engineering

1. **CI/CD**：建立 GitHub Actions pipeline（test → lint → eval）
2. **重构 tasks.py**：拆分为 state_manager / graph_runner / api_routes 三个模块
3. **API 文档**：使用 OpenAPI/Swagger 完善 API 文档
4. **配置管理**：引入 Pydantic Settings 替代手动 .env 解析

16.9 Software Testing

1. **E2E 测试扩展**：覆盖 research workflow 的完整 7 节点路径
2. **Mutation Testing**：引入 mutmut 或 cosmic ray 基础框架
3. **性能测试**：添加 latency / throughput 基准测试
4. **Flaky Test 检测**：建立 flaky test 监控和重跑机制

附录：分析依据的文件清单

核心架构文件

- `docs/current-architecture-and-usage.md`
- `src/graph/state.py`
- `src/graph/builder.py`
- `src/research/graph/builder.py`
- `src/research/agents/supervisor.py`

Agent 实现文件

- `src/research/agents/clarify_agent.py`
- `src/research/agents/search_plan_agent.py`
- `src/research/agents/planner_agent.py`
- `src/research/agents/retriever_agent.py`
- `src/research/agents/analyst_agent.py`
- `src/research/agents/reviewer_agent.py`

工具与 MCP

- `src/tools/registry.py`
- `src/tools/mcp_adapter.py`
- `src/tools/specs.py`

Skills 系统

- `src/skills/registry.py`
- `src/skills/orchestrator.py`
- `src/skills/discovery.py`
- `src/skills/research_skills.py`

Memory 系统

- `src/memory/manager.py`

评测系统

- `src/eval/rag/runner.py`
- `src/eval/rag/metrics.py`

数据库与持久化

- `src/db/models.py`
- `src/db/task_persistence.py`

API 层

- `src/api/routes/tasks.py`
- `src/api/routes/agents.py`
- `src/api/routes/mcp.py`
- `src/api/routes/skills.py`

Trace 系统

- `src/tasking/trace_wrapper.py`
- `src/graph/callbacks.py`

测试文件

- `tests/test_e2e.py`
- `tests/test_e2e_clarify.py`
- `tests/test_integration_cli.py`
- `tests/eval/rag/test_runner.py`
- `tests/research/agents/test_supervisor.py`